

Data Mining Assignment 3 Report

- Author: Juiyun Chang (314551145)
- Github Link: https://github.com/pinkypacman/DM_Asg3
- How to reproduce the results:

See README.md in the provided GitHub repo for the exact execution steps.

In short, the two final Kaggle submissions are regenerated bit-for-bit from cached model outputs using `python reproduce/build_finals.py`. Training-from-scratch commands are also available in the README.md. Every number/figure in this report is reproduced by `python analysis/eda.py` and `python analysis/ablation.py`.

Final Kaggle submissions:

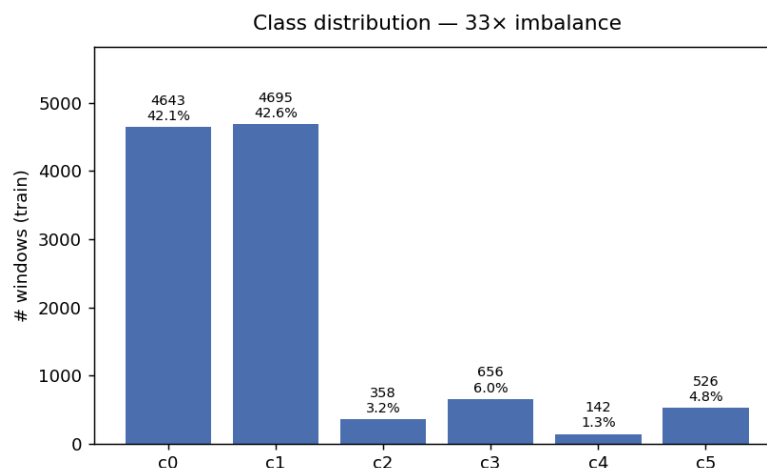
File	Strategy	Public Score
<code>submission/submission_tcn_gru.csv</code>	strong 12-model blend + public-tuned rare-class calibration	0.8160
<code>submission/submission_tcn_robust_v5dist.csv</code>	same blend + distribution-robust decision layer (private bet)	0.8115

Q1. Preliminary analysis (EDA) that informed the design

Data shape & protocol. 60 train users (11,020 windows) and 40 *disjoint* test users (6,849 windows). Because users do not overlap, all validation uses `GroupKFold(user)`. A random split would leak per-user signal and badly over-estimate scores. The label is constant within a file; we collapse each CSV to a `(300, 6)` array.

1.1 Severe Class Imbalance → marco-F1 score will be deeply influenced by the rare classes

Classes 0 and 1 dominate; class 4 is **33x rarer** than class 1. Because the metric is *macro*-F1, the four minority classes (2,3,4,5) weigh as much as the two majority classes, the main bottleneck of this problem will be finding the rare classes accurately.

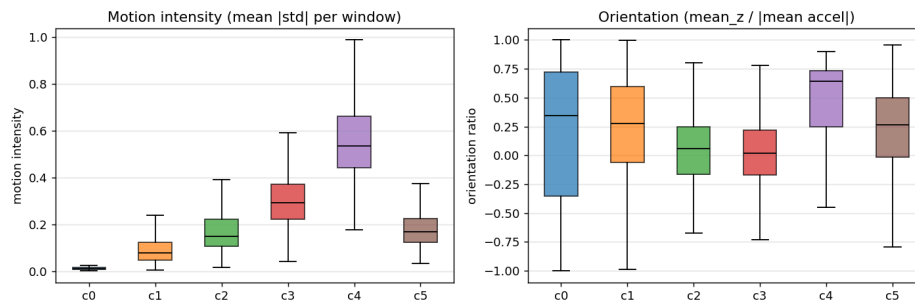


class	0	1	2	3	4	5
count	4643	4695	358	656	142	526
share	42.1%	42.6%	3.25%	5.95%	1.29%	4.77%

1.2 What physically distinguishes the activities?

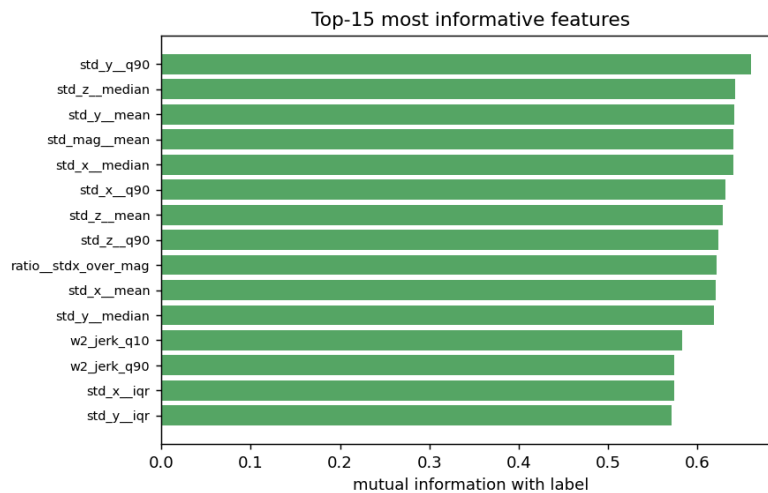
The labels are activity-intensity levels. Two interpretable descriptors computed directly from the raw signals cleanly separate most classes.

- **Motion intensity:** Mean of the per-second `|std|` magnitude.
- **Orientation:** How upright the device is (i.e., `mean_z / |mean accel|`).



Median motion intensity forms a clean ladder **c0 (0.010) < c1 (0.079) < c2 (0.150) ≈ c5 (0.169) < c3 (0.292) < c4 (0.536)**.

Class 0 is still, Class 4 is the most vigorous. This is consistent with the feature-importance ranking:



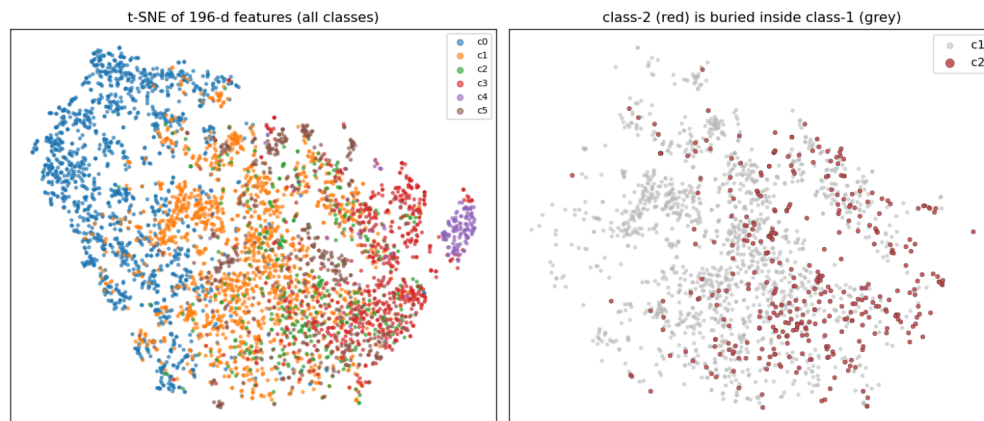
The `std` channels carry the label, while the `mean` (orientation) channels matter less. Every one of the top-8 features by mutual information is a `std`-channel statistic (`std_y_mean`, `std_z_median`, ...).

Therefore, I built features around the `std` channels and motion-magnitude signals, adding orientation as a secondary cue.

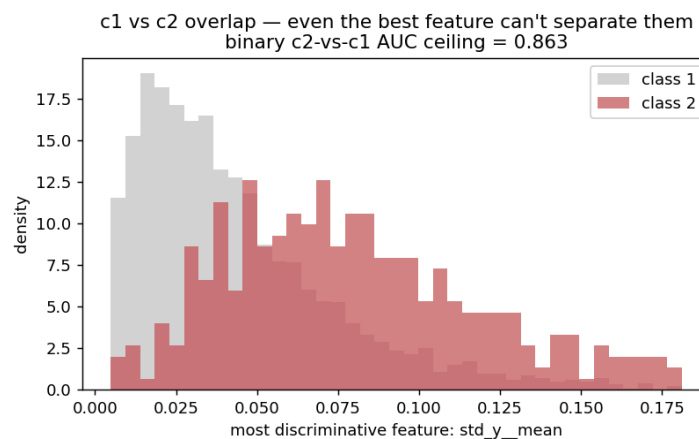
1.3 The Class 2 bottleneck

The descriptors above expose the assignment's central obstacle. Class 2's motion sits between Class 1 and Class 5, and its orientation is near zero, which makes it have no descriptor of its own.

A t-SNE of the full 196-d feature makes this concrete: classes form coherent islands, but **class 2 (red) is scattered inside the class-1 cloud (grey)** rather than forming its own cluster.



Even the single most discriminative feature for a *binary* class-2-vs- class-1 task has tiny mutual information and its histograms overlap heavily, and a tuned Gradient Boosting Decision Tree (GBDT) on that binary problem tops out at **AUC $\approx$ 0.863**. A information ceiling that caps class-2 F1 at roughly 0.4–0.5, no matter the model:



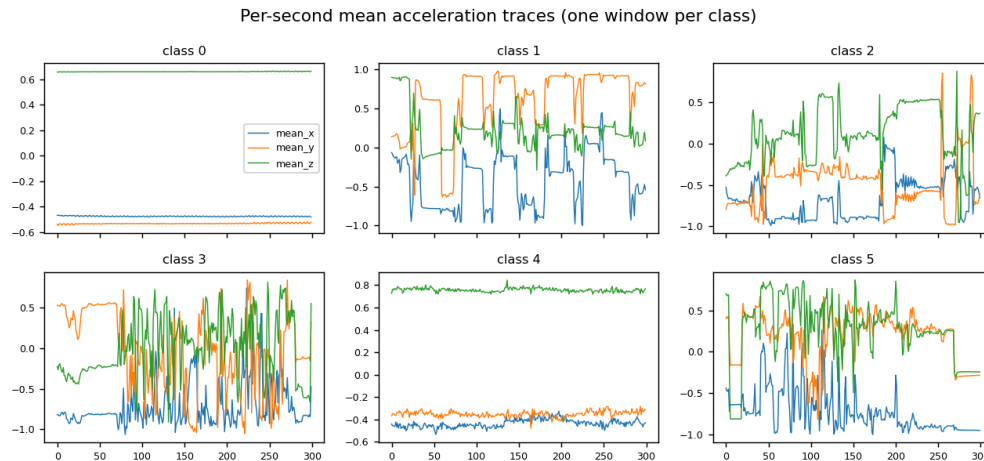
This finding drove the whole strategy: since no model can crack class 2 from these 1 Hz aggregates, the gains must come from **diversity + careful rare-class calibration**.

1.5 Naive baseline results

baseline (GroupKFold Out Of Fold (OOF) macro-F1)	macro-F1	takeaway
Majority class (always c0)	0.0996	Macro-F1 punishes ignoring rare classes
Logistic reg. on the 6 window-mean channels	0.4883	Mean orientation alone is informative (posture)
Position-only (normalized file_id rank)	0.2271	Position nails c0/c1 but is blind to rare classes

baseline (GroupKFold Out Of Fold (OOF) macro-F1)	macro-F1	takeaway
Single LightGBM on 34 hand-stats (Q2)	0.6948	Hand-stats + GBDT is the right backbone

The jump from 0.488 (6 means) to 0.695 (34 stats + GBDT) showed that the summary statistics of the window, not the raw 300-step series, carry most of the signal.



Design implications

- Build a rich hand-feature + GBDT backbone centered on the `std` /motion channels.
- Handle imbalance with balanced weighting and decision-threshold calibration, not resampling that destroys majority classes.
- Validate strictly by user.
- Exploit the temporal/position structure as features
- Accept the class-2 ceiling and chase gains through model diversity.

Q2. Preprocessing techniques and their measured improvement

Preprocessing pipeline:

1. Window assembly & cleaning

Parse each CSV to `(300,6)` float32, verify 300 rows and a constant label, clamp spurious negative `std` to 0, assert finiteness.

2. GroupKFold-by-user fold assignment

Random KFold OOF over-scores by leaking users; GroupKFold is what makes our OOF track the leaderboard ranking.

3. Feature engineering in four cumulative views

I measured each by training one fixed LightGBM configuration, so the deltas isolate the features:



feature set	dim	macro-F1	Δ	class-2 F1
lean: per-channel mean/std/min/max/median + magnitude + orientation	34	0.6948	—	0.239
+ spectral + temporal dynamics + higher-order	116	0.7098	+0.0150	0.227
+ jerk / RMS (orientation-invariant motion: $\text{RMS}=\sqrt{(\text{mean}^2+\text{std}^2)}$, mean-channel jerk)	196	0.7177	+0.0079	0.235
+ position features (normalized <code>file_id</code> rank + neighbours)	201	0.7194	+0.0017	0.199

Each feature group adds macro-F1, with diminishing returns:

- spectral + temporal dynamics give the biggest lift (+0.015).
- jerk/RMS another +0.008.

All spectral, temporal dynamics, and jerk/RMS help the easy classes but never crack class 2. On the other hand, position features actively hurt class 2 by leaking the "rare-classes-live-in-the-middle" prior, leading to over-prediction of c0/c1.

I therefore froze the final GBDT members at the 196-d "richest" set (no position features), this is exactly what `gdbt_richest.npz` / `xgb_richest.npz` in the final blend use. This is also our central empirical result: class 2 is at the data ceiling of the 1 Hz-aggregated features, and it is the macro-F1 bottleneck.

Q3. Aligning labels with the sequential accelerometer readings

The label describes the *whole* window, so I interpret the aligning as **capturing within-window temporal structure** and the **between-window sequence**.

1. Within-window temporal features fed to the GBDT/tabular models:

- **Spectral:**
FFT band powers over `[0,0.05,0.15,0.30,0.51]` Hz plus spectral entropy/centroid/dominant frequency, on `mean_x/y/z` and the magnitude signal.
- **Temporal dynamics:**
Zero-crossing rate, first-difference (jerk) energy, and autocorrelation at lags 1 and 5, these directly encode how the signal evolves second-to-second.
- **Temporal segmentation:**

The 300 s window is split into 5×60 s segments and per-segment statistics are compared, capturing within-window change.

- **Jerk / RMS:**

Derivative-of-mean and $\sqrt{(\text{mean}^2 + \text{std}^2)}$, orientation-invariant motion descriptors.

2. Sequence models on the raw (300, 6) series

Instead of hand features, four deep models read the ordered series and learn temporal dependencies end-to-end:

- **TCN** (dilated causal 1-D convolutions) → 128-d embedding → KNN-on-embedding.
- **1-D ResNet**: residual conv stack
- **Transformer encoder**: self-attention over the 300 steps.
- **Bidirectional recurrent**

These purely-temporal models are competitive with the hand-feature GBDTs despite seeing no engineered statistics.

ResNet reaches **0.7051** OOF and BiGRU **0.6903** (vs LightGBM 0.7193), This is evidence that they genuinely capture the sequential signal, and they make different errors, which is why they add value to the ensemble method I used.

Q4. Ablation study of the core design choices

- Individual members of the ensemble

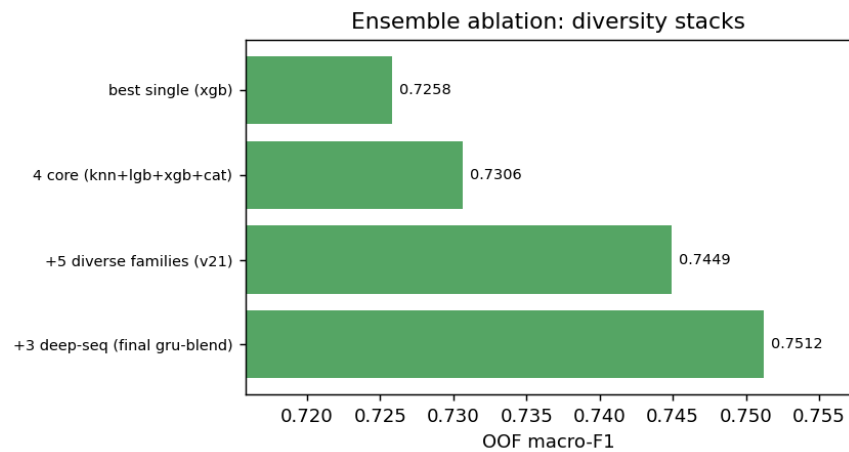
model	OOF macro	c2-F1	family
LightGBM (richest)	0.7193	0.217	boosting
XGBoost (richest)	0.7258	0.265	boosting
CatBoost	0.6932	0.250	boosting
TCN → KNN	0.6769	0.216	conv-embedding
1-D ResNet	0.7051	0.223	conv-sequence
BiGRU	0.6903	0.254	recurrent
Transformer	0.6486	0.258	attention
Inception-Time	0.6587	0.251	multiscale conv
feature-MLP	0.6871	0.234	MLP
bagging-trees	0.6708	0.000	bagging
linear / Nyström-RBF	0.6761	0.241	kernel
TabNet	0.6401	0.218	attention-tabular

- Ensemble construction

Model diversity is where the real gains are.

Adding decorrelated families that each make different errors lifts OOF far more than any single stronger model. On the leaderboard this same progression took us from 0.8008 → 0.8101 → 0.8160, confirming the OOF ordering holds on Kaggle.

stage	OOF macro	Δ
best single model (XGBoost)	0.7258	—
4 core models (TCN-KNN + LGB + XGB + CatBoost)	0.7306	+0.0048
+ 5 diverse families (bagging, MLP, linear, Inception, TabNet)	0.7449	+0.0143
+ 3 deep-sequence models (ResNet, Transformer, BiGRU) → final gru-blend	0.7512	+0.0063



- Rare-class calibration

Decision rule	OOF macro	c2-F1	predicted shares (c0..c5)
raw argmax	0.7512	0.318	—
+ v5 frozen biases	0.7353	0.306	—
+ c2/c5 share-calibration (the gru rule)	0.7282	0.334	[43.2, 41.0, 3.3, 7.7, 1.1, 3.6]
+ prior-match to train prior (the robust rule)	0.7102	0.321	[42.2, 42.5, 3.4, 6.0, 1.3, 4.7]

On same-distribution OOF, raw argmax wins; the calibration rules lower OOF macro. Yet on Kaggle the calibrated submissions **win**: **gru** = 0.8160 and **robust_v5dist** = 0.8115 both beat the un-calibrated/early recipes.

The reason is that OOF measures fit to the training class distribution, while the leaderboard rewards matching the test/private distribution and a well-calibrated rare-class operating point.

I verified the train ↔ test windows are not distribution-shifted, so OOF threshold-search over-fits the rare classes, which is exactly why a principled, low-variance rule generalizes better. Hence the two finals: **gru** is tuned to the public-leader board optimum; **robust_v5dist** trades public score for a distribution that I expect to hold up on the hidden private split.